ABSTRACT

Finding safe, affordable and verified accommodation near the campus is one of the biggest challenges faced by students who move to a new city for higher education. Existing options such as brokers, scattered classified posts and social media groups often hide the real rent, deposit and distance from college. This project presents **Stayora**, a web-based student housing platform that helps students discover verified PGs, single rooms, shared rooms, flats, hostels and roommates close to their college.

Stayora is developed using HTML5, CSS3 and JavaScript, along with third-party services: Firebase and EmailJS for one-time password (OTP) authentication, Leaflet for map-based location display, and Tesseract.js for OCR-based student ID verification. A student verifies identity through an OTP, searches by college or location, filters stays by occupancy, category and maximum rent, and narrows results by distance (2, 5, 10 or 25 km) using the Haversine formula. Students can save favourites, chat with hosts, schedule visits and post roommate requests, while owners can publish listings through a four-step form with photos and a map pin.

The project demonstrates how core web development concepts, namely DOM manipulation, event handling, form validation, modular JavaScript and API integration, can be applied to a real-world problem. The system is lightweight, runs in any modern browser, and is designed to be extended in future with a full backend, a database, online booking and a mobile application.

CONTENT

Table of Contents

Abstract ii

Content iii

Acronyms iv

Nomenclature v

List of Figures vi

List of Tables vii

Chapter 1: Introduction 1

Chapter 2: Literature Survey 3

Chapter 3: Problem Statement 4

Chapter 4: Objectives 5

Chapter 5: Methodology 6

Chapter 6: Implementation 9

Chapter 7: Software and Hardware Requirements 12

Chapter 8: Results and Discussion 13

Chapter 9: Conclusion and Future Scope 15

References 16

ACRONYMS

API	Application Programming Interface
CSS	Cascading Style Sheets
DOM	Document Object Model
HTML	HyperText Markup Language
IDE	Integrated Development Environment
JS	JavaScript
OCR	Optical Character Recognition
OTP	One-Time Password
PG	Paying Guest accommodation
SMS	Short Message Service
UI	User Interface

NOMENCLATURE

d	Great-circle distance between a property and the campus (km)
R	Mean radius of the Earth (6371 km)
φ1, φ2	Latitude of the campus and of the property (radians)
λ1, λ2	Longitude of the campus and of the property (radians)
Δφ, Δλ	Difference in latitude and in longitude (radians)
₹	Indian Rupee, the currency used for rent and deposit

LIST OF FIGURES

Figure 5.1: Workflow of the Stayora system..... 7

LIST OF TABLES

Table 5.1: Project milestones of the Stayora project 8

Table 6.1: Main modules of Stayora and the functions implementing them..... 9

Table 7.1: Software requirements 12

Table 7.2: Hardware requirements..... 12

Table 8.1: Test scenarios and expected behaviour..... 13

CHAPTER 1: INTRODUCTION

1.1 Background

In today's competitive education landscape, thousands of students move to a new city every year to study, and finding safe, affordable and verified accommodation near the campus is one of their biggest challenges. **Stayora** is a web-based student housing platform that helps students discover verified PGs, single rooms, shared rooms, flats, hostels and roommates close to their college. The primary goal of this project is to design and develop a simple, transparent and user-friendly housing portal using web technologies.

The importance of such an application lies in its ability to save students from broker dependence, scattered listings and unclear pricing, since every listing shows its rent, deposit, amenities and walking distance to college upfront. By developing Stayora, students gain an understanding of how theoretical web development concepts can be applied to solve real-world problems. The project incorporates OTP-based authentication, map-based location search, data storage, modular JavaScript programming and logical decision-making to deliver relevant results.

1.2 Scope of the Project

Stayora is a front-end centric web application that runs entirely in a modern browser. Its scope covers the following areas:

- Verified OTP login using a phone number or an email address.
- Searching stays by college or location, occupancy, category and maximum monthly rent.
- Distance-based filtering from the campus (2, 5, 10 or 25 km) and map-based viewing of a property using Leaflet.
- Saving favourites, scheduling visits, in-app chat with hosts and posting roommate requests.
- Posting a new listing through a four-step form with photos and a map pin, and managing one's own listings.
- Support for multiple interface languages and an optional backend API for synchronising listings.

1.3 Organization of the Report

Chapter 2 reviews existing approaches and the technologies used. Chapter 3 states the problem and Chapter 4 lists the objectives. Chapter 5 explains the methodology, workflow and project milestones, while Chapter 6 describes the implementation of each module. Chapter 7 lists the software and hardware requirements, Chapter 8 presents the results and discussion, and Chapter 9 concludes the report with the future scope.

CHAPTER 2: LITERATURE SURVEY

2.1 Existing Ways of Finding Student Accommodation

Students currently search for accommodation through local brokers, word of mouth, notice boards, classified advertisements and social media groups. These sources are fragmented: the same room may be advertised by several people at different prices, details such as deposit and amenities are often missing, and there is rarely any proof that a listing is genuine. General-purpose property portals exist, but they are mostly designed for families and long-term tenants rather than for students who need a room close to a particular college for a few months to a year.

2.2 Technologies Reviewed

- **HTML5, CSS3 and JavaScript** form the foundation of the application and are documented extensively in [1], [2] and [4].
- **Leaflet** [3] is a lightweight open-source JavaScript library for interactive maps. It is used to show a property's location and to let owners drop a pin while posting a listing.
- **Firebase Phone Authentication** [5] provides SMS-based OTP verification, and **EmailJS** [6] allows OTP emails to be sent directly from the browser.
- **Tesseract.js** [7] is a JavaScript port of the Tesseract OCR engine and makes it possible to read text from a student ID card image in the browser.
- **The Haversine formula** [8] calculates the great-circle distance between two points on a sphere from their latitude and longitude, and is a standard method for nearby-search features.

2.3 Research Gap

The review shows that no simple, student-focused and transparent solution combines verified login, distance from college, in-app communication, visit scheduling and roommate search in a single lightweight platform. Stayora is designed to fill this gap using freely available web technologies.

CHAPTER 3: PROBLEM STATEMENT

3.1 Problem Definition

Students who relocate to a new city for education struggle to find safe, affordable and verified accommodation near their campus. Information is scattered across brokers and social media posts, pricing is unclear, listings cannot be trusted, and there is no easy way to compare options, contact owners or schedule a visit from one place.

3.2 Key Challenges

- Dependence on brokers, which adds cost and reduces transparency.
- Scattered listings with hidden rent, deposit and amenity details.
- No reliable way to judge the distance of a stay from the college.
- Difficulty in verifying that a user or a listing is genuine.
- No single place to chat with the host, schedule a visit and find a roommate.

3.3 Proposed Solution

The proposed solution is Stayora, a web portal in which every listing displays rent, deposit, amenities and distance to college upfront. Access is protected by OTP verification, results can be filtered by type, category, rent and distance, and students can complete the whole journey (search, compare, chat, schedule a visit and find a roommate) without a broker.

CHAPTER 4: OBJECTIVES

4.1 Main Objective

The objective of the Stayora project is to create a reliable web application that helps students find verified PGs, rooms, flats and roommates near their college with ease. The project aims to build logical and analytical thinking skills by implementing web development fundamentals in a real-world context. The main goal is to help students search, compare and schedule visits to stays quickly without relying on brokers or random social media posts.

4.2 Specific Objectives

- To design a simple, interactive and user-friendly interface where students can log in with an OTP, search by college or location, and filter stays by occupancy, category and maximum rent.
- To implement modular JavaScript functions for listing properties, saving favourites, scheduling visits, chatting with hosts and posting roommate requests.
- To apply the Haversine formula for showing the stays nearest to the campus within a chosen radius.
- To explore third-party services and libraries such as Leaflet maps, Firebase authentication and OCR-based student ID verification.
- To understand how data is processed, validated and stored in a browser environment.

4.3 Long-Term Objective

The long-term objective is to create a scalable platform that can be extended with a full backend, mobile apps and booking features, thereby bridging the gap between classroom learning and real-world applications of software engineering principles.

CHAPTER 5: METHODOLOGY

5.1 Development Approach

The project was developed using an iterative, module-by-module approach. The work began with requirement analysis and a study of how students actually search for accommodation. A responsive user interface was then designed, followed by the JavaScript logic for each feature, integration of the third-party services, and testing in different browsers. Each module (login, search, listings, visits, chat and roommates) was built and tested separately before being combined into the complete website.

5.2 System Workflow

The workflow of the Stayora project describes the sequence of steps through which the system processes user input and delivers relevant results. It represents the logical and functional flow of data within the website, ensuring smooth navigation and correct output. The workflow begins when the student opens the website and is asked to verify their identity by entering their name, phone number or email, and a one-time password (OTP). After successful verification, the home page unlocks, where the student can enter a college or location, choose occupancy and category, and set the maximum monthly rent.

Once the search is submitted, the system validates the inputs and filters the available listings by type, category and rent. A distance filter based on the Haversine formula can also show the stays nearest to the campus within 2, 5, 10 or 25 km. The student can then open a listing, save it to favourites, view its location on a map, message the host through in-app chat, or schedule a visit by choosing a date and time. Finally, the host can approve the visit request and the student can track its status under My Visit Requests. Students who want to rent out a room can post a listing through a four-step form with photos and a map pin. If a required field is missing or the OTP is wrong, an error message is shown.

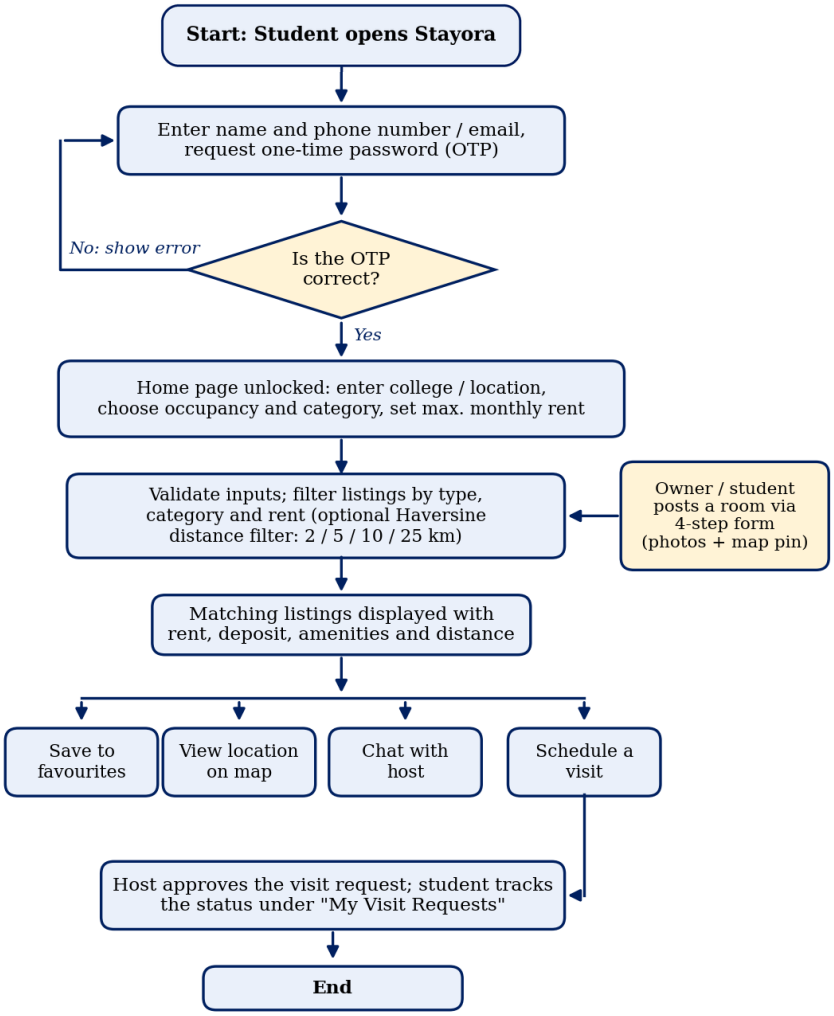


Figure 5.1: Workflow of the Stayora system

5.3 Logic and Algorithms Used

OTP verification. A one-time password is generated and sent to the phone number or email entered by the student. The entered code is compared with the issued one; access is granted only on a match, otherwise an error is displayed and the student may request a new OTP after a resend timer.

Search and filtering. The listings are filtered using conditional logic on the property type (single room, shared PG, boys PG, girls PG, co-living, 1BHK/2BHK flat or roommate needed), the occupancy and the maximum rent entered by the student.

Distance calculation. For every property with known coordinates, the distance from the campus is calculated using the Haversine formula:

$$d = 2R \cdot \arcsin\left(\sqrt{\left(\sin^2(\Delta\varphi/2) + \cos\varphi_1 \cdot \cos\varphi_2 \cdot \sin^2(\Delta\lambda/2)\right)}\right)$$

where R = 6371 km. Only the properties whose distance is within the selected radius (2, 5, 10 or 25 km) are shown.

5.4 Project Milestones

The work was carried out in stages, and each milestone was recorded during the project. Table 5.1 summarises the milestones of the Stayora project.

Table 5.1: Project milestones of the Stayora project

Milestone	Work Completed	Date
M1	Implementation plan: project overview, technology stack, implementation phases and development workflow of the Stayora platform	18 August 2026
M2	Front-end prototype: landing page, search interface, property categories, featured property cards, client-side student login with demo OTP and scroll animations	25 August 2026
M3	Complete frontend development: design system, property discovery and roommate interface, favourites, authentication/OTP screens, multi-step List Your Property modal and responsive layout	2 September 2026
M4	Backend and data handling: Firebase phone OTP and EmailJS email OTP, localStorage sessions, listing create/edit/delete with ownership checks, visit scheduling, private chat, profile and language persistence	9 September 2026
M5	Integration of major features: Leaflet maps and Haversine campus distance, OCR-based student ID verification, four-step listing form, visit approval, Socket.io chat and Node/Express/MongoDB API with JWT	16 September 2026

CHAPTER 6: IMPLEMENTATION

6.1 Overview

Stayora is implemented as a single-page web application using HTML5, CSS3 and JavaScript. HTML structures the pages and modal windows, CSS provides the responsive design and animations, and JavaScript handles the logic, validation, filtering and data storage. The code is organised into small functions, each responsible for one task, which makes the project modular and easy to extend. Table 6.1 lists the main modules and the functions that implement them.

Table 6.1: Main modules of Stayora and the functions implementing them

Module	Purpose	Key functions
Authentication	OTP login by phone or email and session handling	sendOtp, verifyOtp, genOtp, startResendTimer, unlockSite, checkExistingSession
Search and filter	Search by college, occupancy, category and rent	runSearch, applyFilter, clearFilter, parseRent
Map and distance	Show locations and nearby stays	initLpLocationMap, dropPin, haversineDistance, fetchNearby
Listings	Post, edit and delete listings	renderStep, validateStep, handleFiles, publishListing, openModalForEdit, deleteStoredListing
Favourites	Save and view favourite stays	toggleFavourite, getFavIds, saveFavIds, renderFavourites
Visits	Schedule and track visit requests	openVisitModal, submitVisit, saveVisit, setVisitStatus, refreshMyVisits
Chat	Message hosts from within the app	startChatWithHost, sendMessage, renderThreadMessages, refreshUnreadBadge
Roommates	Post and browse roommate requests	openRoommateModal, publishRoommateRequest, renderRoommateCard
ID verification	Read a student ID card using OCR	handleIdVerification (Tesseract.js)
Language	Change the interface language	setLang, applyTranslations, openLanguageModal

6.2 Authentication Module

When the website is opened, the student is asked to enter a name and a phone number or email address. A six-digit OTP is sent using Firebase Phone Authentication (with a reCAPTCHA

verifier) for phone numbers, or using EmailJS for email addresses. When the service keys are not configured, the application falls back to a demonstration (mock) OTP so that the complete flow can still be tested. After the correct OTP is entered, the session is stored in the browser, the home page is unlocked and the student's name appears in the user menu. A resend timer prevents repeated OTP requests.

6.3 Search and Filter Module

The search panel accepts a college or location, an occupancy type, a category (Boys, Girls or Coliving) and the maximum monthly rent. The runSearch function validates these inputs, shows an error message when a required value is missing, and then filters the listing cards by property type, category and rent. The student can also choose a distance of 2, 5, 10 or 25 km from the campus to see only the nearest stays.

6.4 Map and Distance Module

Leaflet is used to display a property's location on an interactive map and to let owners drop a pin on the map while posting a listing. The campus location is stored as a coordinate pair, and the haversineDistance function calculates the distance between the campus and each property. The result is displayed on the listing as "km from campus" and is used for the distance filter.

6.5 Listing Management Module

Students or owners who want to rent out a room use a four-step form: basic details, rent and amenities, photos, and a map pin with a final review. The validateStep function checks the required fields of each step before the next one is shown, and the handleFiles function previews the uploaded photos. On submission, publishListing stores the listing and prepends it to the listing grid. Under My Listings, the owner can edit or delete a posted listing.

6.6 Favourites, Visit Scheduling, Chat and Roommates

A student can mark any listing as a favourite, and the saved stays are shown in a separate list. To schedule a visit, the student selects a date and a time slot; the request is stored and shown under My Visit Requests, and the host can approve it from the requests inbox. The in-app chat lets the student message the host, shows unread-message badges and a typing indicator. The roommate module allows a student to post a roommate request, which is shown as a card for other students to browse.

6.7 Student ID Verification

To increase trust, Stayora includes an optional OCR-based student ID verification. The uploaded ID card image is read in the browser using Tesseract.js, and the extracted text is checked for relevant details. Since the image is processed on the user's device, no separate server is required for this feature.

6.8 Data Storage and Optional Backend

Application data such as the logged-in session, listings, favourites, chats, visit requests, roommate requests and the chosen language is kept in the browser's localStorage, so the project works without any server. For future extension, the code also contains an apiFetch helper that can load and synchronise listings from a backend API, which allows the same front end to be connected to a database later.

CHAPTER 7: SOFTWARE AND HARDWARE REQUIREMENTS

The successful development and execution of the Stayora project depend on selecting appropriate software tools and hardware configurations. The software and hardware requirements are intentionally kept simple to ensure accessibility and compatibility with most modern computers and smartphones.

7.1 Software Requirements

Table 7.1: Software requirements

Component	Details
Programming languages	HTML5, CSS3 and JavaScript. HTML structures the pages, CSS provides the responsive design and animations, while JavaScript handles the logic, validation, filtering and data storage.
Editor / IDE	For coding: Visual Studio Code or Sublime Text. These provide code highlighting, live preview and browser developer tools.
Browser (testing)	Google Chrome, Microsoft Edge or Firefox.
Operating system	Any modern browser on Windows, Linux, macOS or Android.
Libraries and services	Leaflet (maps), Firebase (phone OTP), EmailJS (email OTP) and Tesseract.js (ID scan).

7.2 Hardware Requirements

Table 7.2: Hardware requirements

Component	Minimum requirement
Processor	Intel Core i3 or higher (or equivalent AMD processor)
RAM	Minimum 4 GB
Storage	At least 100 MB free space for the editor and project files
Input / output devices	Keyboard, mouse, monitor and an internet connection

These requirements are minimal, making the project accessible to students with basic computing resources. The simplicity ensures that the website runs on almost any device with a browser while providing hands-on experience in web development, data processing and real-world software development.

CHAPTER 8: RESULTS AND DISCUSSION

8.1 Results

The Stayora website was successfully developed and runs in a modern browser without any installation. The following outcomes were achieved:

- Students can log in securely with an OTP and are shown the home page only after verification.
- Stays can be searched by college or location and filtered by occupancy, category, maximum rent and distance from the campus.
- Each listing displays its rent, deposit, amenities and distance from college upfront, removing the need for brokers.
- Students can save favourites, view a stay on a map, chat with the host and schedule a visit, and can track the visit status.
- Owners can post a listing with photos and a map pin through a four-step form, and edit or delete it later.
- Students can post roommate requests and the interface can be switched between languages.

8.2 Test Scenarios

The application was tested with typical valid and invalid inputs. Table 8.1 lists the main scenarios and the behaviour expected from the system.

Table 8.1: Test scenarios and expected behaviour

Scenario	Input	Expected behaviour
Login with wrong OTP	Valid number, incorrect OTP	Error message is shown; home page stays locked
Login with correct OTP	Valid number, correct OTP	Home page is unlocked
Search with missing field	College left empty	Error message asks for the missing field
Search by rent	Maximum rent entered	Only stays within the rent are listed
Distance filter	Radius of 2 km selected	Only stays within 2 km of the campus are shown
Save favourite	Click on the heart icon	Stay is added to the favourites list
Schedule a visit	Date and time selected	Request appears under My Visit Requests
Post a listing	All four steps completed	Listing is published and appears in the grid

Scenario	Input	Expected behaviour
Post a listing (invalid)	Required field left empty	Next step is blocked and an error is shown

8.3 Discussion

The results show that the main goal of the project, a simple and transparent housing portal for students, has been met. Because the platform uses conditional logic, event handling and form validation, invalid inputs are caught early and the student always receives clear feedback. The use of ready-made services such as Leaflet and Firebase kept the code small while still providing map and OTP features.

The current version also has some limitations. Since data is stored in the browser's localStorage, listings, chats and visit requests are not shared between different devices unless a backend is connected. The OTP features depend on valid Firebase and EmailJS keys, and the OCR-based ID check depends on the clarity of the uploaded image. These limitations are addressed in the future scope.

CHAPTER 9: CONCLUSION AND FUTURE SCOPE

9.1 Conclusion

The Stayora project successfully demonstrates how web development concepts can solve practical problems. It allows students to search for verified PGs, rooms, flats and hostels near campus, save favourites, schedule visits, chat with hosts and find roommates, using HTML, CSS, JavaScript and conditional logic. The project enhances understanding of DOM manipulation, event handling, form validation and modular design. It is user-friendly, accurate and easily extendable to include more cities or features. This project bridges theoretical learning and practical application, providing a hands-on experience in software development.

9.2 Future Scope

- A full backend with a database so that listings, chats and visits are shared across devices.
- Online booking and payment integration for rent and deposit.
- A mobile application for Android and iOS.
- Verified-host badges, reviews and ratings for each stay.
- Support for more cities and colleges, and push or email notifications for visit updates.

REFERENCES

[1] J. N. Robbins, Learning Web Design: A Beginner's Guide to HTML, CSS, JavaScript, and Web Graphics. Sebastopol, CA, USA: O'Reilly Media.

[2] MDN Web Docs, Mozilla. [Online]. Available: https://developer.mozilla.org

[3] Leaflet: an open-source JavaScript library for interactive maps. [Online]. Available: https://leafletjs.com

[4] GeeksforGeeks, Online resources and tutorials. [Online]. Available: https://www.geeksforgeeks.org

[5] Google, "Firebase Authentication: Phone Auth on the web." [Online]. Available: https://firebase.google.com/docs/auth/web/phone-auth

[6] EmailJS, "Send email directly from client-side JavaScript." [Online]. Available: https://www.emailjs.com

[7] Tesseract.js, "Pure JavaScript OCR for multiple languages." [Online]. Available: https://tesseract.projectnaptha.com

[8] R. W. Sinnott, "Virtues of the Haversine," Sky and Telescope, vol. 68, no. 2, p. 159, 1984.